

Systematic review & semantic analysis of literature: CNF dewatering best practices and pontentialities for biomedical applications

Bibliometric and NLP analysis — Source: Scopus - WoS - PubMed

✓ Supplementary Material — Computational Pipeline

Libraries installation

```
1 !pip install -q pyzotero bertopic sentence-transformers umap-learn hdbscan \
2     transformers torch plotly matplotlib pandas numpy tqdm \
3     scikit-learn scipy gensim nltk kaleido
4 !pip install --upgrade pyzotero
5 !pip install bertopic
6 !pip install kaleido
7 !pip install adjustText
8 !pip install upsetplot
9 !pip install plotly==5.24.1 kaleido==0.2.1
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.23->upsetplot) (2023.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=0.23->upsetplot) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=2.0)
Building wheels for collected packages: upsetplot
  Building wheel for upsetplot (pyproject.toml) ... done
```

Literature review and screening through semantic models benchmarking and UMAP best hyperparameters

```
1 """
2 TEMPO-CNF Literature Analysis Pipeline
3 =====
4 Author:      Alfredo E. Villadiego del Villar
5 Affiliation: Universitat de Girona - Lepamap / prodis
6 Date:        June 2026
7 Purpose:     Systematic literature review with NLP (BERTopic, sentiment analysis)
8              and advanced topic evaluation (C_v coherence, diversity, silhouette).
9              Includes UMAP hyperparameter tuning and domain-model benchmarking.
10 =====
11 """
12
13 import os, re, time, warnings, zipfile, itertools
14 from pathlib import Path
15 from collections import defaultdict
16 import pandas as pd
17 import numpy as np
18 from tqdm import tqdm
19 from pyzotero import Zotero
20 from bertopic import BERTopic
21 from sentence_transformers import SentenceTransformer
22 from sklearn.feature_extraction.text import CountVectorizer
23 from umap import UMAP
24 from hdbscan import HDBSCAN
25 from transformers import pipeline, AutoTokenizer, AutoModel
26 import torch
27 from adjustText import adjust_text
28 import seaborn as sns
29 import matplotlib.pyplot as plt
30 import plotly.graph_objects as go
31 import plotly.express as px
32 from typing import List, Dict, Any, Tuple
33 import nltk
34 from nltk.stem import WordNetLemmatizer
35 from gensim.corpora import Dictionary
36 from gensim.models import CoherenceModel
37 from sklearn.metrics import silhouette_score
38
39 # Ensure NLTK data is present
40 nltk.download('wordnet', quiet=True)
41 nltk.download('omw-1.4', quiet=True)
42
43 # -----
44 # CONFIGURATION
45 # -----
46 SEED = 42
47 np.random.seed(SEED)
48 import random
49 random.seed(SEED)
50
51 OUTPUT_DIR = Path('TEMPO_Review_Outputs')
52 OUTPUT_DIR.mkdir(exist_ok=True)
53
54 try:
55     from google.colab import userdata, files
56     IN_COLAB = True
57 except ImportError:
58     IN_COLAB = False
59
60 warnings.filterwarnings('ignore')
61 os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
62
63 plt.rcParams['font.family'] = 'sans-serif'
64 plt.rcParams['font.sans-serif'] = ['Arial', 'DejaVu Sans', 'Helvetica', 'I
65 plt.rcParams['font.size'] = 10
66 plt.rcParams['pdf.fonttype'] = 42
67 plt.rcParams['ps.fonttype'] = 42
68
```



McAfee | WebAdvisor

Your download's being scanned.
We'll let you know if there's an issue.

```

69 # -----
70 # UMAP HYPERPARAMETER GRID (for tuning)
71 # -----
72 UMAP_PARAM_GRID = {
73     'n_neighbors': [15, 20, 25],
74     'min_dist': [0.0, 0.05, 0.1],
75     'n_components': [5, 7, 10]
76 }
77
78 # -----
79 # UTILITY FUNCTIONS
80 # -----
81 def normalize_string(text: str) -> str:
82     if not isinstance(text, str):
83         return ""
84     import unicodedata
85     text = ''.join(c for c in unicodedata.normalize('NFD', text)
86                    if unicodedata.category(c) != 'Mn')
87     text = text.lower()
88     text = re.sub(r'^a-z0-9\\s]', '', text)
89     return re.sub(r'\\s+', ' ', text).strip()
90
91 def normalize_doi(doi: str) -> str:
92     return doi.lower().strip() if isinstance(doi, str) else ""
93
94 def extract_technique_from_paths(paths: List[List[str]]) -> str:
95     for path in paths:
96         if path and any(folder.strip().lower().endswith(' - tempo') for folder in path):
97             return 'TEMPO'
98     return 'Control'
99
100 def extract_macro_theme_from_paths(paths: List[List[str]]) -> str:
101     for path in paths:
102         if path:
103             for folder in path:
104                 if folder.strip().lower().endswith(' - tempo'):
105                     return folder[:-8].strip()
106             return path[-1].strip()
107     return 'Unknown'
108
109 # -----
110 # ZOTERO DATA EXTRACTION
111 # -----
112 class ZoteroCollectionFetcher:
113     def __init__(self, library_id: str, api_key: str):
114         self.zot = Zotero(library_id, 'user', api_key)
115         self.key_to_path = {}
116
117     def build_collection_tree(self):
118         top_colls = self.zot.collections(limit=100)
119         stack = [(c, [], []) for c in top_colls if isinstance(c, dict)]
120         visited = set()
121         while stack:
122             coll, parent, path_so_far = stack.pop()
123             if not isinstance(coll, dict):
124                 continue
125             key = coll.get('key')
126             if key in visited:
127                 continue
128             visited.add(key)
129             name = coll.get('data', {}).get('name', 'Unknown')
130             full_path = path_so_far + [name]
131             self.key_to_path[key] = full_path
132             try:
133                 subs = self.zot.collections(key, limit=100)
134                 for sub in subs:
135                     if isinstance(sub, dict):
136                         stack.append((sub, None, full_path))
137             except Exception:
138                 pass
139         return self.key_to_path
140
141     def fetch_all_items(self) -> List[Dict]:
142         all_items = []
143         start = 0
144         limit = 50
145         while True:
146             topv:

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

147         items = self.zot.items(limit=limit, start=start,
148                                params={'include': 'collections'})
149         if not items:
150             break
151         for item in items:
152             if not isinstance(item, dict):
153                 continue
154             data = item.get('data', {})
155             item_key = data.get('key')
156             coll_keys = data.get('collections', [])
157             paths = [self.key_to_path.get(ck, []) for ck in coll_keys if ck in self.key_to_path]
158             enriched = {
159                 'key': item_key,
160                 'title': data.get('title', ''),
161                 'doi': data.get('DOI', ''),
162                 'abstract': data.get('abstractNote', ''),
163                 'publication_year': data.get('date', ''),
164                 'authors': data.get('creators', []),
165                 'item_type': data.get('itemType', ''),
166                 'collection_paths': paths,
167                 'collection_keys': coll_keys,
168             }
169             all_items.append(enriched)
170             start += limit
171         except Exception:
172             start += limit
173             time.sleep(1)
174     return all_items
175
176
177 class ZoteroDataProcessor:
178     def __init__(self, items: List[Dict]):
179         self.items = items
180
181     def classify_items(self) -> pd.DataFrame:
182         records = []
183         for item in self.items:
184             paths = item.get('collection_paths', [])
185             technique = extract_technique_from_paths(paths)
186             macro_theme = extract_macro_theme_from_paths(paths)
187             records.append({
188                 'key': item['key'],
189                 'title': item['title'],
190                 'doi': item['doi'],
191                 'abstract': item['abstract'],
192                 'publication_year': item['publication_year'],
193                 'authors': item['authors'],
194                 'item_type': item['item_type'],
195                 'technique': technique,
196                 'macro_theme': macro_theme,
197                 'collection_paths': paths
198             })
199         return pd.DataFrame(records)
200
201     def deduplicate_items(self, df: pd.DataFrame):
202         df_dedup = df.copy()
203         df_dedup['doi_norm'] = df_dedup['doi'].apply(normalize_doi)
204         df_dedup['title_norm'] = df_dedup['title'].apply(normalize_string)
205
206         mask_doi = df_dedup['doi_norm'] != ''
207         dup_doi = df_dedup[mask_doi].duplicated(subset='doi_norm', keep=False)
208         for doi, group in df_dedup[mask_doi & dup_doi].groupby('doi_norm'):
209             if (group['technique'] == 'TEMPO').any():
210                 df_dedup = df_dedup.drop(group[group['technique'] != 'TEMPO'].index)
211
212         no_doi = df_dedup['doi_norm'] == ''
213         dup_title = df_dedup[no_doi].duplicated(subset='title_norm', keep=False)
214         for title, group in df_dedup[no_doi & dup_title].groupby('title_norm'):
215             if (group['technique'] == 'TEMPO').any():
216                 df_dedup = df_dedup.drop(group[group['technique'] != 'TEMPO'].index)
217
218         df_dedup = df_dedup.drop_duplicates(subset='title_norm', keep='first')
219         df_dedup = df_dedup.drop(columns=['doi_norm', 'title_norm'])
220         return df_dedup
221
222     def filter_by_abstract(self, df: pd.DataFrame) -> pd.DataFrame:
223         return df[df['abstract'].notna() & (df['abstract'].str.len() > 0)].copy()

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

224
225 def extract_publication_year(self, df: pd.DataFrame) -> pd.DataFrame:
226     def get_year(x):
227         if pd.isna(x) or not isinstance(x, str):
228             return None
229         m = re.search(r'\b(19|20)\d{2}\b', str(x))
230         return int(m.group()) if m else None
231     df['publication_year'] = df['publication_year'].apply(get_year)
232     return df
233
234 def process_pipeline(self) -> pd.DataFrame:
235     df = self.classify_items()
236     df = self.deduplicate_items(df)
237     df = self.filter_by_abstract(df)
238     df = self.extract_publication_year(df)
239     return df
240
241
242 # -----
243 # EMBEDDING MODEL LOADER (supporting both general and domain-specific)
244 # -----
245 def load_embedding_model(model_name: str = 'all-mpnet-base-v2'):
246     """Load sentence transformer or HuggingFace model for embeddings."""
247     if 'matscibert' in model_name.lower() or 'matbert' in model_name.lower():
248         tokenizer = AutoTokenizer.from_pretrained(model_name)
249         model = AutoModel.from_pretrained(model_name)
250         def encode_fn(texts, batch_size=32):
251             all_embeddings = []
252             for i in range(0, len(texts), batch_size):
253                 batch = texts[i:i+batch_size]
254                 encoded = tokenizer(batch, padding=True, truncation=True, return_tensors='pt', max_length=512)
255                 with torch.no_grad():
256                     outputs = model(**encoded)
257                     attention_mask = encoded['attention_mask']
258                     embeddings = (outputs.last_hidden_state * attention_mask.unsqueeze(-1)).sum(1) / attention_mask.sum(1)
259                     all_embeddings.append(embeddings.cpu().numpy())
260             return np.vstack(all_embeddings)
261         return encode_fn
262     else:
263         model = SentenceTransformer(model_name)
264         return lambda texts, batch_size=32: model.encode(texts, show_progress_bar=False, batch_size=batch_size, convert_to
265
266 # -----
267 # TOPIC MODELING PIPELINE
268 # -----
269 class TopicModelingPipeline:
270     def __init__(self, df: pd.DataFrame, seed: int = SEED):
271         self.df = df.copy()
272         self.seed = seed
273         self.model = None
274         self.embeddings = None
275         self.umap_embeddings = None
276         self.coherence_scores = {}
277         self.cv_coherence = None
278         self.topic_diversity = None
279         self.silhouette_highdim = None
280         self.silhouette_umap = None
281
282     def prepare_documents(self) -> List[str]:
283         return self.df['abstract'].fillna('').apply(lambda x: re.sub(r'\s+', ' ', x).strip()).tolist()
284
285     def compute_embeddings(self, documents: List[str], model_name='all-mpnet-base-v2') -> np.ndarray:
286         embed_fn = load_embedding_model(model_name)
287         self.embeddings = embed_fn(documents)
288         return self.embeddings
289
290     def train_topic_model(self, documents: List[str], embeddings: np.ndarray,
291                          umap_kwargs: dict = None):
292         if umap_kwargs is None:
293             umap_kwargs = {'n_neighbors': 15, 'n_components': 5, 'min_dist': 0.0}
294         umap_model = UMAP(metric='cosine', random_state=self.seed, **umap_kwargs)
295         hdbscan_model = HDSCAN(min_cluster_size=12, min_samples=3,
296                                cluster_selection_method='eom', prediction
297         vectorizer_model = CountVectorizer(stop_words='english', ngram_ra
298                                            max_features=10000, min_df=2,
299         model = BERTopic(embedding_model=None,
300                          umap_model=umap_model,
301                          hdbscan_model=hdbscan_model)

```



McAfee | WebAdvisor

Your download's being scanned.
We'll let you know if there's an issue.

```

301         nubscan_model=nubscan_model,
302         vectorizer_model=vectorizer_model,
303         language='english',
304         calculate_probabilities=True,
305         nr_topics=20,
306         verbose=False)
307     topics, _ = model.fit_transform(documents, embeddings)
308     self.model = model
309     self.umap_embeddings = self.model.umap_model.transform(embeddings)
310     return model
311
312     def handle_outliers(self, documents: List[str]):
313         try:
314             self.model.reduce_outliers(documents, self.model.topics_,
315                                     strategy='c-tf-idf', threshold=0.1)
316         except Exception:
317             pass
318
319     def compute_metrics(self, documents: List[str]):
320         # Simple coherence (co-occurrence of top 5 words)
321         topics = set(self.model.topics_) - {-1}
322         for tid in topics:
323             words = [w for w, _ in self.model.get_topic(tid)[:5]]
324             cooc = sum(1 for doc in documents if sum(1 for w in words if w.lower() in doc.lower()) > 1)
325             self.coherence_scores[tid] = cooc / len(documents) if documents else 0.0
326
327         # Advanced metrics
328         tokenized_docs = [doc.lower().split() for doc in documents]
329         dictionary = Dictionary(tokenized_docs)
330         topic_words = []
331         for tid in set(self.model.topics_) - {-1}:
332             words = [w for w, _ in self.model.get_topic(tid)[:10]]
333             topic_words.append(words)
334         coherence_model = CoherenceModel(topics=topic_words,
335                                         texts=tokenized_docs,
336                                         dictionary=dictionary,
337                                         coherence='c_v')
338         self.cv_coherence = coherence_model.get_coherence()
339         all_words = [w for words in topic_words for w in words]
340         unique_words = set(all_words)
341         self.topic_diversity = len(unique_words) / len(all_words) if all_words else 0.0
342
343         # Silhouette scores
344         topics_arr = np.array(self.model.topics_)
345         valid_idx = np.where(topics_arr != -1)[0]
346         if len(valid_idx) >= 2 and len(np.unique(topics_arr[valid_idx])) >= 2:
347             self.silhouette_highdim = silhouette_score(self.embeddings[valid_idx], topics_arr[valid_idx], metric='euclidean')
348             if self.umap_embeddings is not None:
349                 umap_2d = self.umap_embeddings[valid_idx, :2]
350                 self.silhouette_umap = silhouette_score(umap_2d, topics_arr[valid_idx], metric='euclidean')
351         else:
352             self.silhouette_highdim = np.nan
353             self.silhouette_umap = np.nan
354
355     def get_topic_labels(self, n: int = 5) -> Dict[int, str]:
356         lemmatizer = WordNetLemmatizer()
357         labels = {-1: "Not Grouped (Noise)"}
358         for tid in set(self.model.topics_) - {-1}:
359             topic_words = self.model.get_topic(tid)
360             if not topic_words:
361                 labels[tid] = f"Topic_{tid}"
362                 continue
363             lemma_weights = defaultdict(float)
364             for word, weight in topic_words:
365                 lemma = lemmatizer.lemmatize(word.lower(), pos='n')
366                 lemma_weights[lemma] += weight
367             sorted_lemmas = sorted(lemma_weights.items(), key=lambda x: x[1], reverse=True)[:n]
368             labels[tid] = ' + '.join([lemma for lemma, _ in sorted_lemmas])
369         return labels
370
371     def tune_umap_params(self, documents: List[str], embeddings: np.ndarray,
372                         param_grid: dict = None) -> dict:
373         if param_grid is None:
374             param_grid = UMAP_PARAM_GRID
375         keys = param_grid.keys()
376         best_score = -1
377         best_params = None
378         results = []

```



McAfee | WebAdvisor

Your download's being scanned.
We'll let you know if there's an issue.

```

379     for values in itertools.product(*param_grid.values()):
380         params = dict(zip(keys, values))
381         print(f"Testing UMAP params: {params}")
382         umap_model = UMAP(metric='cosine', random_state=self.seed, **params)
383         reduced = umap_model.fit_transform(embeddings)
384         clusterer = HDBSCAN(min_cluster_size=12, min_samples=3, cluster_selection_method='eom')
385         topics = clusterer.fit_predict(reduced)
386         valid = topics != -1
387         if np.sum(valid) > 1 and len(np.unique(topics[valid])) > 1:
388             score = silhouette_score(reduced[valid], topics[valid], metric='euclidean')
389         else:
390             score = -1
391         results.append((params, score))
392         print(f" Silhouette score: {score:.4f}")
393         if score > best_score:
394             best_score = score
395             best_params = params
396     print(f"Best UMAP params: {best_params} with silhouette {best_score:.4f}")
397     self.best_umap_params = best_params
398     return best_params
399
400 def train_pipeline(self, model_name='all-mpnet-base-v2', tune_umap=True):
401     docs = self.prepare_documents()
402     self.compute_embeddings(docs, model_name=model_name)
403     if tune_umap:
404         best_params = self.tune_umap_params(docs, self.embeddings)
405     else:
406         best_params = {'n_neighbors': 15, 'n_components': 5, 'min_dist': 0.0}
407     self.train_topic_model(docs, self.embeddings, umap_kwargs=best_params)
408     self.handle_outliers(docs)
409     self.compute_metrics(docs)
410     labels = self.get_topic_labels()
411     try:
412         self.model._hierarchical_topics = self.model.hierarchical_topics(docs)
413     except Exception:
414         self.model._hierarchical_topics = None
415     return self.model, labels
416
417
418 # -----
419 # BENCHMARKING: Compare general vs domain-specific models
420 # -----
421 def benchmark_models(df, model_names: List[str] = ['all-mpnet-base-v2', 'm3hrdafi/matbert']):
422     results = []
423     for model_name in model_names:
424         print(f"\n--- Benchmarking model: {model_name} ---")
425         pipe = TopicModelingPipeline(df)
426         model, labels = pipe.train_pipeline(model_name=model_name, tune_umap=False)
427         results.append({
428             'model': model_name,
429             'C_v_coherence': pipe.cv_coherence,
430             'topic_diversity': pipe.topic_diversity,
431             'silhouette_highdim': pipe.silhouette_highdim,
432             'silhouette_umap': pipe.silhouette_umap,
433             'num_topics': len(set(model.topics_) - {-1}),
434             'noise_ratio': sum(1 for t in model.topics_ if t == -1) / len(model.topics_)
435         })
436     benchmark_df = pd.DataFrame(results)
437     benchmark_df.to_csv(OUTPUT_DIR / 'Model_Benchmark.csv', index=False)
438     print(benchmark_df)
439     return benchmark_df
440
441 # -----
442 # SENTIMENT ANALYSIS
443 # -----
444 class SentimentAnalysisPipeline:
445     def __init__(self, df: pd.DataFrame):
446         self.df = df.copy()
447         self.model = None
448
449     def load_model(self):
450         try:
451             self.model = pipeline("text-classification",
452                                   model="lxyuan/distilbert-base-multiling
453                                   batch_size=16,
454                                   device=0 if torch.cuda.is_available() el
455         except Exception:

```



McAfee | WebAdvisor

Your download's being scanned.
We'll let you know if there's an issue.

```

456         self.model = None
457
458     def analyze_sentiments(self) -> pd.DataFrame:
459         abstracts = self.df['abstract'].fillna('').tolist()
460         sentiments = []
461         if self.model:
462             for i in range(0, len(abstracts), 32):
463                 batch = [t[:512] for t in abstracts[i:i+32]]
464                 preds = self.model(batch)
465                 for p in preds:
466                     label = p['label'].lower()
467                     if 'pos' in label:
468                         sentiments.append('positive')
469                     elif 'neg' in label:
470                         sentiments.append('negative')
471                     else:
472                         sentiments.append('neutral')
473         else:
474             sentiments = ['neutral'] * len(abstracts)
475         self.df['sentiment'] = sentiments
476         self.df['sentiment_confidence'] = 0.5
477         return self.df
478
479
480 # -----
481 # PUBLICATION-QUALITY VISUALISATIONS
482 # -----
483 class PublicationQualityVisualizations:
484     def __init__(self, df, model, labels, embeddings=None):
485         self.df = df
486         self.model = model
487         self.labels = labels
488         self.embeddings = embeddings
489         self.output_dir = OUTPUT_DIR
490
491     def plot_class_comparison(self, top_n=15):
492         top_topics = self.df['topic_id'].value_counts().head(top_n).index.tolist()
493         data = []
494         for tid in top_topics:
495             data.append({
496                 'topic': self.labels.get(tid, f"T{tid}"),
497                 'TEMPO': ((self.df['topic_id'] == tid) & (self.df['technique'] == 'TEMPO')).sum(),
498                 'Control': ((self.df['topic_id'] == tid) & (self.df['technique'] == 'Control')).sum()
499             })
500         comp = pd.DataFrame(data).sort_values('TEMPO', ascending=True)
501         fig, ax = plt.subplots(figsize=(16,10))
502         y = np.arange(len(comp))
503         width = 0.35
504         ax.barh(y - width/2, comp['TEMPO'], width, label='TEMPO',
505                color='#4A4A4A', edgecolor='black', hatch= '/')
506         ax.barh(y + width/2, comp['Control'], width, label='Control',
507                color='#A9A9A9', edgecolor='black', hatch= '\\\\')
508         ax.set_yticks(y)
509         ax.set_yticklabels(comp['topic'], fontsize=12)
510         ax.set_xlabel('Documents')
511         ax.set_title('Topic Distribution: TEMPO vs Control', fontsize=14, fontweight='bold')
512         ax.legend()
513         plt.tight_layout()
514         plt.savefig(self.output_dir/'Class_Comparison.png', dpi=300)
515         plt.close()
516
517     def plot_sentiment_by_topic(self, top_n=15):
518         top_topics = self.df['topic_id'].value_counts().head(top_n).index.tolist()
519         rows = []
520         for tid in reversed(top_topics):
521             sub = self.df[self.df['topic_id'] == tid]
522             rows.append({
523                 'topic': self.labels.get(tid, f"T{tid}"),
524                 'positive': (sub['sentiment'] == 'positive').sum(),
525                 'neutral': (sub['sentiment'] == 'neutral').sum(),
526                 'negative': (sub['sentiment'] == 'negative').sum()
527             })
528         sent_df = pd.DataFrame(rows).set_index('topic')
529         fig, ax = plt.subplots(figsize=(16,10))
530         colors = ['#4A4A4A', '#A9A9A9', '#2F2F2F']
531         hatches = ['/', '\\\\', '|']
532         left = np.zeros(len(sent_df))
533         for col, color, hatch in zip(['positive', 'neutral', 'negative'], colors, hatches):

```



McAfee | WebAdvisor

Your download's being scanned.
We'll let you know if there's an issue.


```

534         ax.barh(sent_df.index, sent_df[col], left=left, color=color,
535                  edgcolor='black', hatch=hatch, label=col.capitalize())
536         left += sent_df[col].values
537     ax.set_xlabel('Number of Documents', fontsize=14, fontweight='bold')
538     ax.set_title('Sentiment Distribution per Topic', fontsize=14, fontweight='bold')
539     ax.tick_params(axis='y', labels=14)
540     ax.legend()
541     plt.tight_layout()
542     plt.savefig(self.output_dir/'Sentiment_by_Topic.png', dpi=300)
543     plt.close()
544
545     def plot_sentiment_by_class(self):
546         cross = pd.crosstab(self.df['technique'], self.df['sentiment'])
547         for cls in ['TEMPO', 'Control']:
548             if cls not in cross.index:
549                 cross.loc[cls] = 0
550         cross = cross.reindex(['TEMPO', 'Control'])
551         fig, ax = plt.subplots(figsize=(10,6))
552         colors = ['#4A4A4A', '#A9A9A9', '#2F2F2F']
553         hatches = ['//', '\\\\', '|']
554         width = 0.25
555         x = np.arange(len(cross))
556         for i, (col, color, hatch) in enumerate(zip(cross.columns, colors, hatches)):
557             ax.bar(x + i*width, cross[col], width, color=color,
558                  edgcolor='black', hatch=hatch, label=col.capitalize())
559         ax.set_xticks(x + width)
560         ax.set_xticklabels(cross.index)
561         ax.set_xlabel('Class', fontsize=14, fontweight='bold')
562         ax.set_ylabel('Documents', fontsize=14, fontweight='bold')
563         ax.set_title('Sentiment Distribution by Class', fontsize=14, fontweight='bold')
564         ax.tick_params(axis='both', labels=14)
565         ax.legend()
566         plt.tight_layout()
567         plt.savefig(self.output_dir/'Sentiment_by_Class.png', dpi=300)
568         plt.close()
569
570     def create_multipanel_figure(self):
571         import matplotlib.gridspec as gridspec
572         import matplotlib.image as mpimg
573         png_paths = {
574             'class_comp': self.output_dir / 'Class_Comparison.png',
575             'sent_topic': self.output_dir / 'Sentiment_by_Topic.png',
576             'sent_class': self.output_dir / 'Sentiment_by_Class.png'
577         }
578         fig = plt.figure(figsize=(20, 16), dpi=300)
579         gs = gridspec.GridSpec(2, 2, height_ratios=[2, 1], width_ratios=[1, 1])
580         ax1 = fig.add_subplot(gs[0, 0])
581         ax2 = fig.add_subplot(gs[0, 1])
582         ax3 = fig.add_subplot(gs[1, :])
583         for ax, key in zip([ax1, ax2, ax3], ['class_comp', 'sent_topic', 'sent_class']):
584             img = mpimg.imread(png_paths[key])
585             ax.imshow(img)
586             ax.axis('off')
587         labels = ['A', 'B', 'C']
588         for ax, label in zip([ax1, ax2, ax3], labels):
589             ax.text(-0.12, 1.08, label, transform=ax.transAxes,
590                  fontsize=16, fontweight='bold', fontfamily='sans-serif',
591                  va='top', ha='left')
592         plt.tight_layout()
593         fig.savefig(self.output_dir / 'Multipanel_Figure.png', dpi=300, bbox_inches='tight')
594         fig.savefig(self.output_dir / 'Multipanel_Figure.pdf', format='pdf', bbox_inches='tight')
595         plt.close(fig)
596
597     def plot_umap_projection(self):
598         umap_model = self.model.umap_model
599         umap_emb = umap_model.transform(self.embeddings)
600         df_plot = pd.DataFrame({
601             'x': umap_emb[:, 0],
602             'y': umap_emb[:, 1],
603             'topic': self.model.topics_
604         })
605         df_plot = df_plot[df_plot.topic != -1]
606         plt.figure(figsize=(16,10), dpi=300)
607         sns.set_style("white")
608         scatter = sns.scatterplot(data=df_plot, x='x', y='y', hue='topic',
609                                  palette='tab20', s=10, alpha=0.4, legend=
610         texts = []

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

611     for tid in sorted(df_plot.topic.unique()):
612         cx = df_plot[df_plot.topic == tid]['x'].mean()
613         cy = df_plot[df_plot.topic == tid]['y'].mean()
614         label = self.labels.get(tid, f"Topic {tid}")
615         wrapped = "\n".join(label.split(" + ")[0:2])
616         texts.append(plt.text(cx, cy, wrapped, fontsize=12, fontweight='bold',
617                               bbox=dict(facecolor='white', alpha=0.7, edgecolor='none', pad=0.5)))
618     adjust_text(texts, force_text=0.5, expand_points=(1.5,1.5),
619                 arrowprops=dict(arrowstyle='->', color='black', lw=0.5, shrinkA=5, shrinkB=5))
620 plt.title("UMAP Projection of Research Landscapes", fontsize=16, pad=20)
621 plt.xlabel("UMAP Dimension 1")
622 plt.ylabel("UMAP Dimension 2")
623 sns.despine()
624 plt.tight_layout()
625 plt.savefig(self.output_dir/'UMAP_Projection.png', dpi=300, bbox_inches='tight')
626 plt.close()
627
628 def create_interactive_topic_map(self):
629     fig = self.model.visualize_topics()
630     fig.write_html(str(self.output_dir / 'Interactive_Topic_Map.html'))
631     try:
632         fig.write_image(str(self.output_dir / 'Interactive_Topic_Map.png'), scale=3)
633     except Exception:
634         pass
635
636 def plot_temporal_evolution(self, top_n=15):
637     df_y = self.df[self.df['publication_year'].notna()].copy()
638     df_y = df_y[(df_y['publication_year'] >= 2021) & (df_y['publication_year'] <= 2026)]
639     if df_y.empty:
640         return
641     top_topics = self.df['topic_id'].value_counts().head(top_n).index.tolist()
642     rows = []
643     for tid in top_topics:
644         grp = df_y[df_y['topic_id'] == tid].groupby('publication_year').size().reset_index(name='count')
645         grp['topic'] = tid
646         grp['label'] = self.labels.get(tid, f"T{tid}")
647         rows.append(grp)
648     full = pd.concat(rows)
649     dash_styles = ['solid', 'dash', 'dot', 'dashdot', 'longdash', 'longdashdot', '5px 2px 2px 2px', '3px 3px']
650     colors = ['#1a1a1a', '#333', '#4A4A4A', '#666', '#808080', '#A9A9A9', '#C0C0C0', '#D3D3D3']
651     fig = px.line(full, x='publication_year', y='count', color='label', line_dash='label',
652                  color_discrete_sequence=colors, line_dash_sequence=dash_styles)
653     fig.update_layout(
654         title='Topic Evolution Over Time',
655         height=600,
656         font_family='Arial',
657         xaxis=dict(title='Publication Year', tickmode='linear', tick0=2021, dtick=1, range=[2021, 2026]),
658         yaxis=dict(title='Number of Documents')
659     )
660     fig.write_html(str(self.output_dir/'Temporal_Evolution.html'))
661     try:
662         fig.write_image(str(self.output_dir/'Temporal_Evolution.png'), scale=3)
663     except Exception:
664         pass
665     plt.close()
666
667 def generate_all_visualizations(self):
668     self.plot_class_comparison(top_n=15)
669     self.plot_sentiment_by_topic(top_n=15)
670     self.plot_sentiment_by_class()
671     self.plot_temporal_evolution(top_n=15)
672     self.plot_umap_projection()
673     self.create_interactive_topic_map()
674     try:
675         if hasattr(self.model, '_hierarchical_topics') and self.model._hierarchical_topics is not None:
676             fig_dendro = self.model.visualize_hierarchy(
677                 hierarchical_topics=self.model._hierarchical_topics
678             )
679             fig_dendro.write_html(str(self.output_dir / 'Topic_Hierarchy_Dendrogram.html'))
680     except Exception:
681         pass
682     try:
683         heatmap_fig = self.model.visualize_heatmap(width=1200, height=
684         topic_ids = sorted([t for t in set(self.model.topics_) if t !
685         labels_full = [self.labels.get(tid, f"T{tid}") for tid in top
686         heatmap_fig.update_xaxes(tickvals=topic_ids, ticktext=labels_t
687         heatmap_fig.update_yaxes(tickvals=topic_ids, ticktext=labels_full,
688         heatmap_fig.update_layout(

```



McAfee | WebAdvisor

Your download's being scanned.
We'll let you know if there's an issue.

```

689         coloraxis_colorbar=dict(len=1.0, thickness=20, x=1.02, y=0.5,
690                                 yanchor='middle', title='Similarity',
691                                 title_font=dict(size=12, family='Arial, sans-serif')),
692         margin=dict(l=300, r=200, t=80, b=200),
693         width=1200, height=1000,
694         font=dict(size=12, family="Arial, sans-serif")
695     )
696     heatmap_fig.write_html(str(self.output_dir / 'Topic_Similarity_Heatmap.html'))
697 except Exception:
698     pass
699
700
701 # -----
702 # EXPORT UTILITIES
703 # -----
704 class ExportManager:
705     def __init__(self, df, model, labels, coherence_scores, cv_coherence, topic_diversity,
706                 silhouette_highdim, silhouette_umap, output_dir=OUTPUT_DIR):
707         self.df = df
708         self.model = model
709         self.labels = labels
710         self.coherence_scores = coherence_scores
711         self.cv_coherence = cv_coherence
712         self.topic_diversity = topic_diversity
713         self.silhouette_highdim = silhouette_highdim
714         self.silhouette_umap = silhouette_umap
715         self.output_dir = output_dir
716         self.output_dir.mkdir(exist_ok=True)
717
718     def export_main_results(self):
719         export_df = self.df[['title', 'doi', 'topic_id', 'publication_year',
720                             'technique', 'macro_theme', 'sentiment', 'sentiment_confidence']].copy()
721         export_df['topic_label'] = export_df['topic_id'].map(self.labels)
722         export_df.to_csv(self.output_dir/'Supplemental_Material_Topic_Model.csv', index=False)
723
724     def export_topic_representatives(self, n_docs=10):
725         representatives = []
726         for topic_id in sorted(set(self.df['topic_id'])):
727             topic_docs = self.df[self.df['topic_id'] == topic_id].nlargest(n_docs, 'sentiment_confidence')
728             for idx, (_, row) in enumerate(topic_docs.iterrows(), 1):
729                 representatives.append({
730                     'topic_id': topic_id,
731                     'topic_label': self.labels.get(topic_id, ''),
732                     'rank': idx,
733                     'title': row['title'],
734                     'doi': row['doi'],
735                     'sentiment': row['sentiment']
736                 })
737         pd.DataFrame(representatives).to_csv(self.output_dir/'Topic_Representative_Documents.csv', index=False)
738
739     def export_coherence_scores(self):
740         coh = [{'topic_id': tid, 'topic_label': self.labels.get(tid, ''),
741                 'coherence_score': score, 'document_count': (self.df['topic_id'] == tid).sum()}
742               for tid, score in self.coherence_scores.items()]
743         pd.DataFrame(coh).to_csv(self.output_dir/'Topic_Coherence_Scores.csv', index=False)
744
745     def export_advanced_metrics(self):
746         metrics = pd.DataFrame([
747             'C_v_coherence': self.cv_coherence,
748             'Topic_diversity': self.topic_diversity,
749             'Silhouette_high_dimension': self.silhouette_highdim,
750             'Silhouette_from_umap': self.silhouette_umap
751         ])
752         metrics.to_csv(self.output_dir/'Advanced_Topic_Metrics.csv', index=False)
753
754     def export_class_statistics(self):
755         stats = []
756         for tech in ['TEMPO', 'Control']:
757             sub = self.df[self.df['technique'] == tech]
758             if sub.empty:
759                 continue
760             for theme in sub['macro_theme'].unique():
761                 subset = sub[sub['macro_theme'] == theme]
762                 stats.append({
763                     'technique': tech,
764                     'macro_theme': theme,
765                     'document_count': len(subset),

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

766         'unique_topics': subset['topic_id'].nunique(),
767         'positive_pct': round((subset['sentiment'] == 'positive').sum() / len(subset) * 100, 2),
768         'neutral_pct': round((subset['sentiment'] == 'neutral').sum() / len(subset) * 100, 2),
769         'negative_pct': round((subset['sentiment'] == 'negative').sum() / len(subset) * 100, 2),
770         'year_range': f"{int(subset['publication_year'].min())}-{int(subset['publication_year'].max())}"
771     })
772     pd.DataFrame(stats).to_csv(self.output_dir/'Class_Statistics.csv', index=False)
773
774     def export_all(self):
775         self.export_main_results()
776         self.export_topic_representatives()
777         self.export_coherence_scores()
778         self.export_advanced_metrics()
779         self.export_class_statistics()
780
781     def create_zip_archive(self):
782         zip_path = Path('TEMPO_Review_Outputs.zip')
783         with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zipf:
784             for fp in self.output_dir.rglob('*'):
785                 if fp.is_file():
786                     zipf.write(fp, fp.relative_to(self.output_dir.parent))
787         return zip_path
788
789
790 # -----
791 # MAIN EXECUTION
792 # -----
793 def main():
794     if IN_COLAB:
795         ZOTERO_ID = userdata.get('ZOTERO_ID')
796         ZOTERO_KEY = userdata.get('ZOTERO_KEY')
797     else:
798         ZOTERO_ID = input("Zotero ID: ")
799         ZOTERO_KEY = input("Zotero Key: ")
800
801     fetcher = ZoteroCollectionFetcher(ZOTERO_ID, ZOTERO_KEY)
802     fetcher.build_collection_tree()
803     items = fetcher.fetch_all_items()
804
805     processor = ZoteroDataProcessor(items)
806     df = processor.process_pipeline()
807
808     # Benchmark models
809     benchmark_models(df, model_names=['all-mpnet-base-v2', 'm3rg-iitd/matscibert'])
810
811     topic_pipe = TopicModelingPipeline(df)
812     model, labels = topic_pipe.train_pipeline(model_name='all-mpnet-base-v2', tune_umap=True)
813     df['topic_id'] = model.topics_
814
815     sent_pipe = SentimentAnalysisPipeline(df)
816     sent_pipe.load_model()
817     df = sent_pipe.analyze_sentiments()
818
819
820 # =====
821 # COMPUTATIONAL RIGOR SCORE FOR CNF LITERATURE (Applicable to associated research fields)
822 # =====
823
824     # Definition of rigor indicators for CNF / biomaterials research
825
826     rigor_keywords = {
827         # ----- GOOD indicators (+1 each) -----
828         'mechanical_testing': r'\b(mechanical\s+(test|property|strength|modulus)|tensile|stress-strain)\b',
829         'rheological_char': r'\b(rheolog|viscosity|viscoelastic|oscillation|shear\s+rate)\b',
830         'microscopy': r'\b(TEM|AFM|SEM|transmission\s+electron|atomic\s+force|scanning\s+electron)\b',
831         'spectroscopy': r'\b(FTIR|XRD|XPS|NMR|Raman|spectroscop)\b',
832         'chemical_quant': r'\b(carboxyl\s+content|conductometric\s+titration|zeta\s+potential|degree\s+of\s+oxidation)\b',
833         'statistical_analysis': r'\b(statistical\s+(analysis|test)|ANOVA|t-test|p-value|standard\s+deviation|confidence\s+
834         'replicates': r'\b(replicate|triplicate|duplicate|n\s+*\s+d{2,})\b',
835         'control_group': r'\b(control|comparison|reference|blank)\b',
836         'longitudinal': r'\b(time\s+course|follow-up|over\s+time|longitudinal)\b',
837         'validated_method': r'\b(validated|standard\s+protocol|ISO|ASTM)\b',
838
839         # ----- POOR indicators (-1 each) -----
840         'preliminary': r'\b(preliminary|exploratory|pilot|feasibility)\b',
841         'limited_char': r'\b(limited\s+characterization|insufficient\s+dat
842         'no_stats': r'\b(no\s+statistical\s+analysis|without\s+statistics|descriptive\s+only|qualitative\s+assessment)\b',

```



McAfee | WebAdvisor

Your download's being scanned.
We'll let you know if there's an issue.

```

843     'small_sample': r'\b(small\s+sample|n\s+=\s+[1-9]\d+|few\s+samples)\b',
844     'self_report': r'\b(self[- ]report|questionnaire\s+only|subjective)\b',
845     'convenience': r'\b(convenience\s+sample|opportunistic|non-randomized)\b',
846 }
847
848 # Scoring function
849
850 def compute_rigor_score(text: str) -> tuple:
851     """
852     Returns (score, details_dict) where score is raw integer sum.
853     """
854     if not isinstance(text, str):
855         return 0, {}
856     text_lower = text.lower()
857     score = 0
858     details = {}
859
860     # Check GOOD indicators
861     good_keys = ['mechanical_testing', 'rheological_char', 'microscopy', 'spectroscopy',
862                 'chemical_quant', 'statistical_analysis', 'replicates', 'control_group',
863                 'longitudinal', 'validated_method']
864     for key in good_keys:
865         if re.search(rigor_keywords[key], text_lower, re.IGNORECASE):
866             score += 1
867             details[key] = 'Present'
868         else:
869             details[key] = 'Absent'
870
871     # Check POOR indicators
872     poor_keys = ['preliminary', 'limited_char', 'no_stats', 'small_sample',
873                 'self_report', 'convenience']
874     for key in poor_keys:
875         if re.search(rigor_keywords[key], text_lower, re.IGNORECASE):
876             score -= 1
877             details[key] = 'Present (penalty)'
878         else:
879             details[key] = 'Absent'
880
881     return score, details
882
883 # Apply to corpus (title + abstract)
884 df['texto_completo'] = df['title'].fillna('') + " " + df['abstract'].fillna('')
885
886 # Compute scores
887 df['rigor_score'], df['rigor_details'] = zip(*df['texto_completo'].apply(compute_rigor_score))
888
889 # Normalise to 0-100 scale
890 min_score = df['rigor_score'].min()
891 max_score = df['rigor_score'].max()
892 if max_score > min_score:
893     df['rigor_score_norm'] = (df['rigor_score'] - min_score) / (max_score - min_score) * 100
894 else:
895     df['rigor_score_norm'] = 50.0 # neutral if all equal
896
897 # Export rigor scores to CSV
898 rigor_csv_path = OUTPUT_DIR / 'SM_Rigor_Scores_CNF.csv'
899 df[['title', 'doi', 'topic_id', 'technique', 'macro_theme',
900     'rigor_score', 'rigor_score_norm', 'rigor_details']].to_csv(rigor_csv_path, index=False)
901
902
903 # Visualizations
904
905 sns.set_style("whitegrid")
906 plt.rcParams.update({
907     'font.family': 'sans-serif',
908     'font.sans-serif': ['Arial', 'DejaVu Sans', 'Helvetica'],
909     'font.size': 10,
910     'pdf.fonttype': 42,
911     'ps.fonttype': 42,
912 })
913
914 # Average rigor by topic (bar chart)
915 topic_rigor = df.groupby('topic_id').agg(
916     mean_rigor=('rigor_score_norm', 'mean'),
917     count=('rigor_score_norm', 'count'),
918     std_rigor=('rigor_score_norm', 'std')
919 ).reset_index()
920

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```

921 # Exclude noise topic (labeled as "-1" in topics classification)
922 topic_rigor = topic_rigor[topic_rigor['topic_id'] != -1]
923
924 # Map topic labels
925 def get_topic_label(tid):
926     if tid in labels:
927         return labels[tid]
928     else:
929         return f"Topic {tid}"
930
931 topic_rigor['label'] = topic_rigor['topic_id'].apply(get_topic_label)
932
933 # Sort by mean rigor
934 topic_rigor_sorted = topic_rigor.sort_values('mean_rigor', ascending=True)
935
936 fig, ax = plt.subplots(figsize=(12, 6))
937 bars = ax.barh(topic_rigor_sorted['label'], topic_rigor_sorted['mean_rigor'],
938               xerr=topic_rigor_sorted['std_rigor'], capsiz=5,
939               color='darkblue', alpha=0.7, edgecolor='black')
940 ax.set_xlabel('Computational Rigor Score (0-100)', fontsize=12)
941 ax.set_title('Methodological Rigor by Thematic Cluster (CNF Literature)', fontsize=14, fontweight='bold')
942 ax.axvline(x=50, color='red', linestyle='--', linewidth=1.5, label='Moderate Rigor Threshold')
943 ax.legend()
944 plt.tight_layout()
945 rigor_topic_path = OUTPUT_DIR / 'SM_Figure_Rigor_by_Topic_CNF.png'
946 plt.savefig(rigor_topic_path, dpi=300, bbox_inches='tight')
947 plt.close(fig)
948
949 # Rigor distribution by class (TEMPO vs Control)
950 fig, ax = plt.subplots(figsize=(8, 6))
951 sns.boxplot(data=df[df['topic_id'] != -1], x='technique', y='rigor_score_norm',
952            palette={'TEMPO': '#4AAAAA', 'Control': '#A9A9A9'}, ax=ax)
953 sns.swarmplot(data=df[df['topic_id'] != -1], x='technique', y='rigor_score_norm',
954              color='black', alpha=0.4, size=3, ax=ax)
955 ax.set_xlabel('Class', fontsize=12)
956 ax.set_ylabel('Rigor Score (0-100)', fontsize=12)
957 ax.set_title('Rigor Score Distribution by Class', fontsize=14, fontweight='bold')
958 plt.tight_layout()
959 rigor_class_path = OUTPUT_DIR / 'SM_Figure_Rigor_by_Class_CNF.png'
960 plt.savefig(rigor_class_path, dpi=300, bbox_inches='tight')
961 plt.close(fig)
962
963 # Heatmap of rigor indicators - count per topic
964 indicator_cols = [k for k in rigor_keywords.keys() if k not in ['preliminary', 'limited_char', 'no_stats', 'small_sample']]
965
966 for key in indicator_cols:
967     df[f'flag_{key}'] = df['texto_completo'].apply(
968         lambda t: 1 if re.search(rigor_keywords[key], str(t), re.IGNORECASE) else 0
969     )
970
971 # Aggregate per topic
972 topic_indicator = df[df['topic_id'] != -1].groupby('topic_id')[f'flag_{k}' for k in indicator_cols].mean()
973 topic_indicator.index = topic_indicator.index.map(get_topic_label)
974
975 # Plot heatmap
976 fig, ax = plt.subplots(figsize=(14, max(6, 0.4 * len(topic_indicator))))
977 sns.heatmap(topic_indicator, annot=True, fmt='.2f', cmap='Blues', cbar_kws={'label': 'Proportion of documents'},
978            linewidths=0.5, ax=ax)
979 ax.set_title('Prevalence of Rigor Indicators by Topic', fontsize=14, fontweight='bold')
980 ax.set_xlabel('Indicator')
981 ax.set_ylabel('Topic')
982 plt.tight_layout()
983 rigor_heatmap_path = OUTPUT_DIR / 'SM_Figure_Rigor_Heatmap_CNF.png'
984 plt.savefig(rigor_heatmap_path, dpi=300, bbox_inches='tight')
985 plt.close(fig)
986
987 print("\n=== Computational Rigor Scores (CNF) ===")
988 print(f"Mean rigor score (all docs): {df['rigor_score_norm'].mean():.2f} ± {df['rigor_score_norm'].std():.2f}")
989 print(f"Min: {df['rigor_score_norm'].min():.2f}, Max: {df['rigor_score_norm'].max():.2f}")
990 print("\nAverage rigor by topic (top 5):")
991 print(topic_rigor_sorted[['label', 'mean_rigor', 'count']].tail(5).to_string(index=False))
992
993
994 viz = PublicationQualityVisualizations(df, model, labels, embeddings=
995 viz.generate_all_visualizations()
996 viz.create_multipanel_figure()
997

```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.

```
998     exp = ExportManager(df, model, labels, topic_pipe.coherence_scores,
999                          topic_pipe.cv_coherence, topic_pipe.topic_diversity,
1000                          topic_pipe.silhouette_highdim, topic_pipe.silhouette_umap)
1001     exp.export_all()
1002     zip_path = exp.create_zip_archive()
1003
1004     if IN_COLAB:
1005         from google.colab import files
1006         files.download(str(zip_path))
1007
1008 if __name__ == '__main__':
1009     main()
```



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.



McAfee | WebAdvisor



Your download's being scanned.
We'll let you know if there's an issue.